Desarrollo de una API RESTful

Objetivos:

Que demuestren los conocimientos vistos en la materia sobre desarrollo de web APIs, manejo de bases de datos, seguridad, autenticación, y buenas prácticas de desarrollo en Python.

Caso de uso ejemplo:

Desarrollar una API para un sistema de gestión de reservas de un gimnasio.

Los casos de uso son:

- RQ1 - Los usuarios podrán registrarse
- RQ2 - Los usuarios podrán ver horarios
- RQ3 - Los usuarios podrán reservar clases
- RQ4 - Los usuarios podrán cancelar reservas
- RQ5 - Los usuarios podrán recibir notificaciones ante cambios de fechas/horarios

Consigna:

Cada grupo deberá plantear casos de uso similares al planteado arriba para el proyecto elegido, indicando al menos 5 requerimientos de alto nivel, los cuales deberán están contemplados en los endpoints de la API.

Stack mínimo a entregar:

1. Framework para la API (en FastAPI)
2. Autenticación: para el registro y autenticación segura de los usuarios.
3. Base de datos manejada con SQLAlchemy
4. Documentación del proyecto, con la idea general del proyecto, los requerimientos funcionales, y casos de uso.
5. Contenedor en docker de la API
6. Funciones de Testing
7. Deploy

Compartir el repositorio github del proyecto a matuk, santiagoblaslaguzza

Requisitos adicionales:

- La API debe seguir principios RESTful.
- Deben implementarse mecanismos de manejo de errores mediante excepciones.
- Se espera que los estudiantes entreguen un código bien documentado.
- Aplicar al menos un patrón de diseño que tenga sentido y utilidad real.
- Incluir al menos 10 tests unitarios que validan el funcionamiento de la API.
- Incluir una GitHub Action para la automatización de los tests en GitHub
- Proveer un archivo README en el repositorio GitHub con las librerías que necesita la API e instrucciones claras sobre cómo desplegar y probar la API.
- Incluir una presentación indicando:
    - Al menos 1 diagrama UML
    - Dónde, porqué y qué patrones de diseño se utilizaron

Evaluación:

1. Funcionalidad (20%): Si la API cumple correctamente con los requerimientos funcionales y maneja correctamente los errores.
2. Calidad del código (20%): Claridad de código y uso adecuado de conceptos vistos en la materia.
3. Documentación y pruebas (20%): Calidad de la documentación y cobertura de las pruebas.
4. Exposición y defensa (40%): claridad de conceptos y justificación de los conceptos aplicados.